

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Кафедра інформаційних систем та технологій

Тема: 13. Office communicator

Курсова робота

З дисципліни «Технології розроблення програмного забезпечення»

Керівник

доц. Амонс О.А.

«Допущений до захисту»

(Особистий підпис керівника)

« » _____ 2024р.

Захищений з оцінкою

(оцінка)

Члени комісії:

(особистий підпис)

(особистий підпис)

Виконавець

ст. Шадлер А.А.

залікова книжка № 32 – 30

гр. ІА-32

(особистий підпис виконавця)

« » _____ 2024р.

(розшифровка підпису)

(розшифровка підпису)

Київ – 2025

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
(назва навчального закладу)

Кафедра ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ
Дисципліна «Технології розроблення програмного забезпечення»
Курс 3 Група ІА-32 Семестр 1

ЗАВДАННЯ
на курсову роботу студента
Шадлер Андрій Андрійович
(прізвище, ім'я, по батькові)

1. Тема роботи:

Мережевий комунікатор для офісу повинен нагадувати функціонал програми Skype з можливостями голосового / відео / конференц-зв'язку, відправки текстових повідомлень і файлів (можливо, офлайн), веденням організованого списку груп / контактів

2. Строк здачі студентом закінченої роботи _____

3. Вихідні дані до роботи:

Мова програмування: C#, Vue.JS

Середовище програмування: Visual Studio 2022, Visual Studio Code

Функціональні вимоги:

Програма повинна забезпечувати можливість реєстрації та авторизації користувачів, обмін текстовими повідомленнями, голосовими та відео-дзвінками (у тому числі груповими), підтримувати конференц-зв'язок, організований список контактів, збереження історії чатів

Об'єкт розробки:

Програмна система – корпоративний мережевий комунікатор для внутрішнього використання в офісі, який забезпечує текстовий, голосовий та відео-зв'язок між працівниками з підтримкою групового спілкування

4. Зміст розрахунково – пояснювальної записки (перелік питань, що підлягають розробці)

Огляд існуючих рішень

Загальний опис проєкту

Вимоги до застосунків системи

Функціональні вимоги до системи

Нефункціональні вимоги до системи

Сценарії використання системи

Концептуальна модель системи

Вибір бази даних

Вибір мови програмування та середовища розробки

Проектування розгортання системи

Структура бази даних

Архітектура системи

Специфікація системи

Вибір та обґрунтування патернів реалізації

Інструкція користувача

5.Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

- a. Діаграма варіантів^[1]
- b. Концептуальна діаграма компонентів^[2]
- c. Діаграма розгортання системи^[3]

6. Дата видачі завдання 10 вересня 2025 р

КАЛЕНДАРНИЙ ПЛАН

[illegible]

Студент _____
(підпис)

Андрій ШАДЛЕР
(Ім'я ПРИЗВИЩЕ)

Керівник _____
(підпис)

Олександр АМОНС
(Ім'я ПРИЗВИЩЕ)

«31» жовтня 2025 р.

ЗМІСТ

ВСТУП	3
1 ПРОЄКТУВАННЯ СИСТЕМИ.....	4
1.1. Огляд існуючих рішень	4
1.2. Загальний опис проєкту	5
1.3. Вимоги до застосунків системи	5
1.3.1. Функціональні вимоги до системи.....	5
1.3.2. Нефункціональні вимоги до системи	6
1.4. Сценарії використання системи.....	8
1.5. Концептуальна модель системи.....	12
1.6. Вибір бази даних	13
1.7. Вибір мови програмування та середовища розробки.....	14
1.8. Проєктування розгортання системи	14
2 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ	17
2.1. Структура бази даних	17
2.2. Архітектура системи	19
2.2.1. Специфікація системи	19
2.2.2. Вибір та обґрунтування патернів реалізації.....	20
2.3. Інструкція користувача.....	21
ВИСНОВКИ.....	23
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	Помилка! Закладку не визначено.
ДОДАТКИ.....	Помилка! Закладку не визначено.

ВСТУП

У більшості компаній спілкування між працівниками відбувається за допомогою популярних месенджерів. Вони зручні, але не завжди враховують специфіку корпоративного середовища. Наприклад, усі дані зберігаються на сторонніх серверах, тому компанія не має повного контролю над листуванням і файлами. Також складно інтегрувати такі сервіси з внутрішніми системами чи обмежити доступ лише для працівників.

Крім того, у загальних месенджерах відсутні функції, важливі саме для офісної роботи: централізоване адміністрування користувачів, створення груп за відділами, чи захищене збереження історії повідомлень. Через це виникає потреба у власному комунікаторі, який працюватиме у межах корпоративної мережі та враховуватиме вимоги безпеки й зручності.

Метою цієї роботи є створення мережевого комунікатора для офісу, який забезпечить обмін повідомленнями, голосові та відео-дзвінки, а також конференц-зв'язок. Такий застосунок дозволить об'єднати всі необхідні функції в одному середовищі й зробить внутрішню комунікацію більш надійною та організованою.

1 ПРОЄКТУВАННЯ СИСТЕМИ

1.1. Огляд існуючих рішень

На ринку програмного забезпечення існує велика кількість засобів для онлайн-комунікації. Найпоширенішими з них є **Telegram**, **Discord**, **Slack**, **Microsoft Teams** та **Zoom**. Кожен із цих застосунків має власні переваги, проте більшість із них орієнтовані на загальне користування або великі організації, а не на локальне офісне середовище.

Telegram є швидким і зручним месенджером із підтримкою текстових чатів, дзвінків і передачі файлів. Проте він зберігає дані на віддалених серверах і не дозволяє створити повністю автономну мережу для внутрішнього користування. Крім того, Telegram не передбачає централізованого керування користувачами чи поділу їх на групи за відділами.

Discord спочатку створювався для спілкування геймерів, але з часом отримав підтримку серверів, голосових каналів і відеоконференцій. Незважаючи на широкий функціонал, Discord не надає можливості розгорнути власний сервер у корпоративному середовищі без підключення до зовнішніх сервісів, що ускладнює його використання в закритих офісних мережах.

Slack та **Microsoft Teams** орієнтовані саме на командну роботу, проте вони є комерційними продуктами, що вимагають оплати за розширений функціонал. Вони мають потужні можливості для інтеграції з іншими сервісами, проте є надлишково складними для невеликих офісів, де потрібен простий і швидкий засіб спілкування.

Zoom, своєю чергою, спеціалізується переважно на відеоконференціях і не забезпечує зручного постійного обміну повідомленнями та файлами.

Таким чином, аналіз показує, що хоча існуючі рішення дозволяють організувати спілкування між користувачами, вони або не надають достатньої гнучкості, або не забезпечують необхідного рівня безпеки та локальності. Тому доцільно розробити власний мережевий комунікатор, який поєднує базові функції сучасних месенджерів із можливістю автономної роботи всередині корпоративної мережі.

1.2. Загальний опис проєкту

Метою проєкту є створення мережевого комунікатора для внутрішнього використання в офісі. Система має забезпечувати обмін текстовими повідомленнями, голосові та відео-дзвінки, а також підтримувати групові чати й конференції. Основна ідея полягає в тому, щоб об'єднати всі засоби комунікації в одному застосунку.

Програмний продукт складатиметься з клієнтської та серверної частин. Серверна частина, розроблена мовою C# із використанням технологій .NET, відповідатиме за обробку запитів, зберігання даних користувачів, історії повідомлень і керування сеансами зв'язку. Клієнтська частина, створена з використанням TypeScript та фреймворку Vue.js, забезпечуватиме зручний графічний інтерфейс і взаємодію користувача з системою.

Проєкт орієнтований на використання в невеликих і середніх офісах, де потрібен швидкий і захищений зв'язок. Комунікатор дозволитиме працювати в реальному часі, зберігати історію спілкування.

1.3. Вимоги до застосунків системи

1.3.1. Функціональні вимоги до системи

Мережева система комунікації повинна забезпечувати повний набір інструментів для обміну інформацією між користувачами офісу. Основні функціональні вимоги такі:

a. Реєстрація та авторизація користувачів.

Система повинна надавати можливість створення нового облікового запису, авторизації та виходу із системи.

b. Список контактів і груп.

Користувач має бачити перелік своїх контактів і груп, мати змогу додавати нові, редагувати або видаляти наявні

c. Текстовий обмін повідомленнями.

Повідомлення повинні передаватися в реальному часі з можливістю відображення історії чатів

d. Голосовий та відео-зв'язок.

Повинна бути реалізована можливість здійснення індивідуальних дзвінків і групових конференцій

e. Збереження даних.

Вся інформація про повідомлення, дзвінки, файли та користувачів повинна зберігатися в базі даних з можливістю подальшого перегляду

1.3.2. Нефункціональні вимоги до системи

До нефункціональних вимог належать характеристики, що визначають якість роботи системи, її надійність і зручність у використанні:

a. Продуктивність.

Система повинна забезпечувати швидку передачу повідомлень і файлів без значних затримок навіть при одночасній роботі декількох користувачів

b. Безпека.

Дані користувачів мають бути захищені за допомогою шифрування та авторизації. Доступ до системи – лише для зареєстрованих користувачів

c. Надійність.

У разі помилки або збою зв'язку програма повинна зберігати дані та автоматично відновлювати з'єднання після відновлення мережі

d. Масштабованість.

Система має дозволяти додавання нових користувачів і розширення функціоналу без необхідності суттєвих змін у базовій архітектурі

e. Зручність інтерфейсу.

Інтерфейс має бути інтуїтивно зрозумілим, із чіткою структурою меню, повідомлень і кнопок керування

1.4. Сценарії використання системи

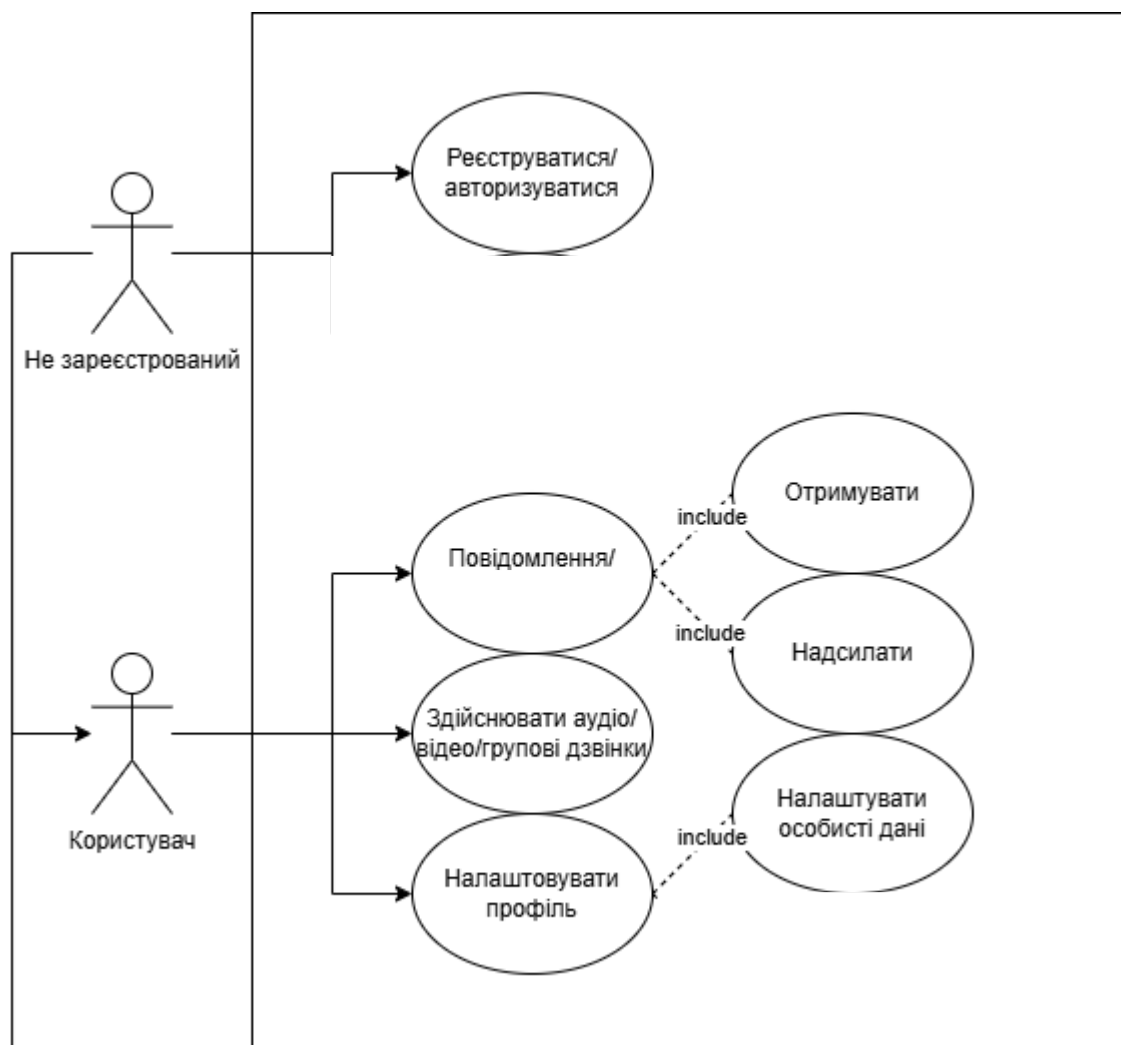


Рис. 1.4.1 Діаграма варіантів

Передумови:

Користувач авторизований у системі та має стабільне підключення до мережі.

Постумови:

Повідомлення доставлено адресату або збережено у черзі, якщо одержувач офлайн.

Взаємодіючі сторони:

Користувач, Система обміну повідомленнями (чат-сервер).

Короткий опис:

Користувач вводить текст повідомлення та надсилає його іншому користувачу або в групу. Система обробляє запит і доставляє повідомлення адресату.

Основний перебіг подій:

- Користувач відкриває чат.
- Користувач вводить повідомлення й натискає «Надіслати».
- Система приймає повідомлення та зберігає його.
- Система відправляє повідомлення одержувачу.
- Одержувач отримує повідомлення в реальному часі.

Винятки:

- №1: Втрата з'єднання. Повідомлення зберігається локально та надсилається пізніше.
- №2: Користувач недоступний (офлайн) – повідомлення потрапляє у чергу.
- №3: Невалідне повідомлення (порожнє або заборонений контент) – система відхиляє надсилання.

Сценарій 2: Здійснення дзвінка

Передумови:

Користувач авторизований і має дозволи на використання камери та мікрофона.

Постумови:

Відеодзвінок завершено, історія дзвінка збережена.

Взаємодіючі сторони:

Користувач (ініціатор), інший користувач (одержувач), Система відеодзвінків.

Короткий опис:

Користувач ініціює відеодзвінок до іншого користувача. Система встановлює з'єднання, передає аудіо- та відеопотоки між учасниками, а після завершення – фіксує результат.

Основний перебіг подій:

- Користувач відкриває список контактів.
- Обирає користувача та натискає «Відеодзвінок».
- Система надсилає виклик одержувачу.
- Одержувач приймає дзвінок.
- Система встановлює з'єднання й починає передачу потоків.
- Один із користувачів завершує дзвінок.
- Система припиняє передачу даних і зберігає історію.

Винятки:

- №1: Проблеми зі з'єднанням – дзвінок автоматично завершується.

Примітки:

Система може знизити якість відео при низькій швидкості мережі.

Сценарій 3: Реєстрація нового користувача

Передумови:

Користувач має доступ до мережі та ще не має облікового запису.

Постумови:

Новий користувач зареєстрований і може входити до системи.

Взаємодіючі сторони:

Незареєстрований користувач, Система реєстрації.

Короткий опис:

Незареєстрований користувач створює обліковий запис, вводячи необхідні дані (логін, пароль, номер телефону, ім'я).

Основний перебіг подій:

- Користувач відкриває форму реєстрації.
- Вводить логін, пароль, ім'я, телефон.
- Система перевіряє коректність введених даних.
- Система створює новий обліковий запис.
- Користувач отримує повідомлення про успішну реєстрацію.

Винятки:

- №1: Користувач уже існує – система повідомляє про помилку.

- №2: Невалідні дані – реєстрація не відбувається.
- №3: Відсутній доступ до сервера – запит не обробляється.

1.5. Концептуальна модель системи

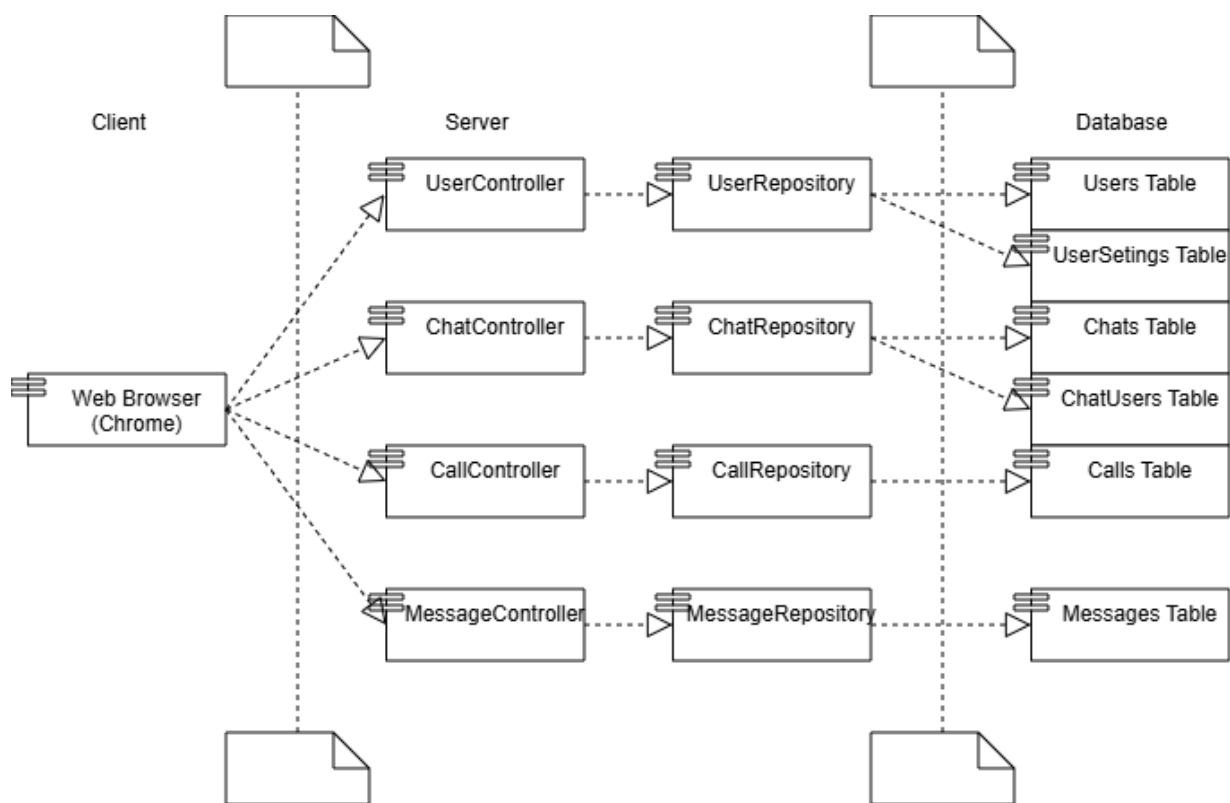


Рис. 1.5.1 Концептуальна діаграма компонентів

Опис:

1. Client(Клієнтська частина):
 - a. Компонент Web Browser (Chrome) відповідає за взаємодію з користувачем. Він відправляє HTTP-запити на сервер і відображає UI-відповіді
2. Server(Серверна частина):
 - a. Контролери (UserController, ChatController, CallController, MessageController) керують репозиторіями (UserRepository, ChatRepository, CallRepository, MessagePerository)
 - b. Репозиторії (UserRepository, ChatRepository, CallRepository, MessagePerository) напряму обробляють запити користувачів і надсилають дані в базу даних
3. Database(База даних):
 - a. Users Table – таблиця для збереження інформації про клієнтів

- b. UserSettings Table – таблиця для збереження інформації про персоналізацію клієнтського інтерфейсу
 - c. Chats Table – таблиця для збереження чатів та їх описів
 - d. ChatUsers Table – поміжна таблиця для збереження приналежності користувачів до чатів
 - e. Calls Table – таблиця для збереження записів про дзвінки
 - f. Messages Table – таблиця для збереження повідомлень
4. Зв'язки:
- a. Web Browser надсилає запити до контроллерів
 - b. Контроллери передають їх у репозиторії
 - c. Репозиторії обробляють запити та надсилають їх у бази даних

1.6. Вибір бази даних

Для розробки мережевого комунікатора було обрано Microsoft SQL Server (MSSQL) як систему керування базами даних. MSSQL забезпечує надійне зберігання інформації, високу продуктивність при обробці великої кількості запитів і можливість інтеграції з технологіями .NET. Вибір MSSQL також обумовлений його широкою підтримкою інструментів для адміністрування, резервного копіювання та контролю доступу користувачів.

Для спрощення роботи з базою даних у проєкті використовується Entity Framework (EF) – ORM (Object-Relational Mapping) технологія, яка дозволяє оперувати даними бази у вигляді об'єктів C# замість написання SQL-запитів вручну. EF підтримує автоматичне створення структури бази даних відповідно до моделей даних, забезпечує відстеження змін та спрощує роботу з відносинами між таблицями.

Комбінація MSSQL і Entity Framework дозволяє реалізувати безпечну, масштабовану та ефективну систему зберігання даних для клієнт-серверного

комунікатора, зменшуючи при цьому кількість ручної роботи при розробці та обслуговуванні бази даних.

1.7. Вибір мови програмування та середовища розробки

Для розробки мережевого комунікатора було обрано C# як основну мову програмування для серверної частини. C# добре інтегрується з платформою .NET, що забезпечує високу продуктивність, надійність і безпеку при обробці запитів, реалізації бізнес-логіки та взаємодії з базою даних. Використання C# також дозволяє легко інтегрувати Entity Framework, що спрощує роботу з базою даних та управління об'єктами даних, а також ASP.NET, який допоможе з налаштуванням маршрутів запитів.

Клієнтська частина розробляється з використанням TypeScript у поєднанні з фреймворком Vue.js. TypeScript забезпечує типізовану та надійну роботу з JavaScript-кодом, а сучасні фронтенд-фреймворки дозволяють створити зручний і інтуїтивний інтерфейс користувача з підтримкою інтерактивних чатів, дзвінків і сповіщень.

Як середовище розробки було обрано Visual Studio 2022 для серверної частини, що забезпечує інтегровану підтримку C#, .NET, Entity Framework і ASP.NET, та Visual Studio Code для клієнтської частини, що дозволяє зручно працювати з TypeScript, Vue.js і підключати різні розширення для фронтенду.

Поєднання обраних мов і середовищ розробки забезпечує гнучкість, масштабованість і стабільну роботу системи як на стороні сервера, так і на стороні клієнта, а також спрощує процес подальшого розвитку і підтримки програмного продукту.

1.8. Проєктування розгортання системи

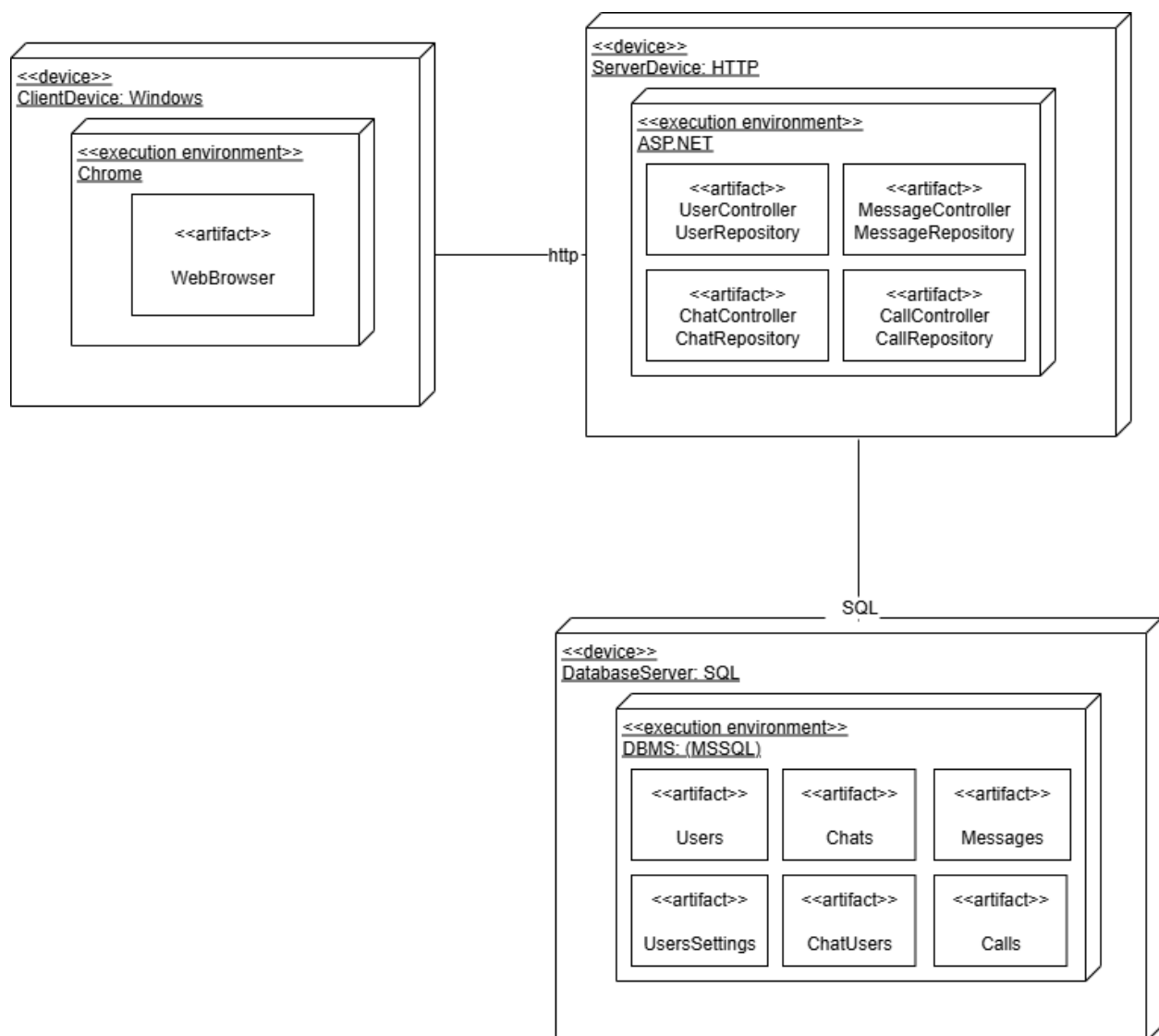


Рис. 1.8.1 Діаграма розгортання системи

Опис:

1. ClientDevice: Windows
 - a. Це фізичний пристрій користувача
 - b. Усередині розгорнуто середовище Chrome (**<<execution environment>>**), яке представляє веб-бравзер
 - c. У бравзері запускається артефакт “Web Browser”, який відповідає за інтерфейс та відправку HTTP запитів до серверу
2. ServerDevice: HTTP
 - a. Серверна машина, що обробляє HTTP-запити

- b. Виконується ASP.NET, у якому розміщені серверні артефакти
- c. UserController -> UserRepository – модуль, що відповідає за обробку запитів пов'язаних із користувачами (створення, авторизація, зміна налаштувань/профілю)
- d. ChatController -> ChatRepository – модуль, що відповідає за обробку запитів пов'язаних із чатом (створення, відображення повідомлень)
- e. MessageController -> MessageRepository, а також CallController -> CallRepository – модулі, що відповідають за збереження історії повідомлень або дзвінків відповідно

3. DatabaseServer: SQL

- a. Це сервер, що відповідає за збереження та обробку даних
- b. Усередині виконується DBMS (MSSQL) як середовище виконання
- c. Users(Table) – таблиця для збереження користувачів
- d. UsersSettings(Table) – таблиця для збереження налаштувань користувачів
- e. Chats(Table) – таблиця для збереження чатів
- f. ChatUsers(Table) – допоміжна таблиця для розв'язку багатовимірних зв'язків між Users та Chats у випадку групових чатів.
- g. Messages(Table) – таблиця для збереження логів повідомлень
- h. Calls(Table) – таблиця для збереження логів дзвінків

4. Зв'язки:

- a. ClientDevice-ServerDevice – взаємодія через HTTP-протокол, для надсилання та отримання запитів і відповідей
- b. ServerDevice-DatabaseDevice – взаємодія через SQL, для створення нових чи пошуку існуючих записів у базі даних

2 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ

2.1. Структура бази даних

Для реалізації системи було обрано реляційну базу даних Microsoft SQL Server, яка забезпечує надійне збереження даних, підтримку зв'язків між таблицями та можливість виконання запитів для отримання потрібної інформації. Основна мета структури бази даних – організувати ефективне зберігання даних про користувачів, чати та повідомлення, а також забезпечити можливість масштабування системи у майбутньому.

Основні таблиці:

1. **Users** – таблиця користувачів системи.

- Id (PK) – унікальний ідентифікатор користувача.
- Name – повне ім'я користувача.
- Username – логін користувача для входу в систему.
- Phone – контактний номер телефону.
- Password – хеш пароля користувача для автентифікації.
- Role – роль користувача.
- Setting (FK) – ідентифікатор налаштувань інтерфейсу
- Chat (FK) – ідентифікатор списку чатів

2. **Chats** – таблиця чатів.

- Id (PK) – унікальний ідентифікатор чату.
- Name – назва чату.
- Description – опис чату або коротка інформація про його призначення.
- User (FK) – ідентифікатор переліку користувачів

3. **UserChats** – таблиця зв'язку «багато до багатьох» між користувачами та чатами.

- UserId (FK) – ідентифікатор користувача.
- ChatId (FK) – ідентифікатор чату.

- Дозволяє визначати, які користувачі беруть участь у конкретному чаті.

4. **Messages** – таблиця повідомлень у чатах.

- Id (PK) – унікальний ідентифікатор повідомлення.
- ChatId (FK) – ідентифікатор чату, до якого належить повідомлення.
- OwnerId (FK) – ідентифікатор користувача, який відправив повідомлення.
- Text – текст повідомлення.
- TimeSend – дата та час відправки повідомлення.

5. **Settings** – таблиця персональних налаштувань користувачів.

- Id (PK) – унікальний ідентифікатор налаштування.
- UserId (FK) – ідентифікатор користувача, якому належать налаштування.
- BgColor – колір фону інтерфейсу чату.
- FontSize – розмір тексту для відображення повідомлень.
- User (FK) – ідентифікатор приналежності до користувача

6. **Calls** – таблиця дзвінків у чатах

- Id (PK) – унікальний ідентифікатор дзвінка.
- ChatId (FK) – ідентифікатор чату, у якому відбувався дзвінок.
- OwnerId (FK) – ідентифікатор користувача, який ініціював дзвінок.
- TimeStart – дата та час початку дзвінка.
- TimeEnd – дата та час завершення дзвінка.
- TimeDuration – тривалість дзвінка (можна обчислювати як різницю TimeEnd - TimeStart).
- Status – статус дзвінка.

Зв'язки між таблицями:

- Users ↔ Messages – один користувач може відправляти багато повідомлень; кожне повідомлення належить лише одному користувачу.
- Chats ↔ Messages – один чат містить багато повідомлень; кожне повідомлення належить лише одному чату.
- Users ↔ Chats – зв'язок багато до багатьох через таблицю UserChats, що дозволяє одному користувачу брати участь у декількох чатах та навпаки.

- Users ↔ Settings – один користувач має одне налаштування (залежно від реалізації, можливо, один-до-одного).
- Chats ↔ Calls – один чат може мати багато дзвінків; кожен дзвінок належить лише одному чату.
- Users ↔ Calls – один користувач може ініціювати багато дзвінків; кожен дзвінок має одного власника.

Пояснення:

Така структура забезпечує логічну організацію даних і дозволяє ефективно виконувати запити для отримання списку чатів, повідомлень конкретного користувача, а також керувати персональними налаштуваннями. Використання зовнішніх ключів (FK) гарантує цілісність даних і запобігає появі «висячих» записів у базі.

2.2. Архітектура системи

Система побудована за клієнт-серверною архітектурою з використанням ASP.NET Core для серверної частини та Vue 3 для фронтенду. Серверна частина відповідає за зберігання даних, автентифікацію користувачів, обробку повідомлень та дзвінків, а клієнтська частина забезпечує інтерфейс для користувачів, відображення чатів, повідомлень та налаштувань.

2.2.1. Специфікація системи

Система включає такі основні компоненти:

1. Серверна частина (Backend)

- Реалізована на ASP.NET Core.
- Забезпечує REST API для роботи з користувачами, чатами, повідомленнями та налаштуваннями.

- Впроваджена автентифікація за допомогою JWT для безпечного доступу до ендпоінтів.
- Використовує Entity Framework Core для роботи з базою даних та ORM-зв'язків між таблицями.

2. Клієнтська частина (Frontend)

- Реалізована на Vue 3 з використанням CDN (без складних build-процесів).
- Забезпечує вхід/реєстрацію користувачів, перегляд списку чатів, надсилання повідомлень, історію дзвінків, а також персональні налаштування (колір фону, розмір тексту).

3. База даних (Database)

- Реляційна структура, що включає таблиці Users, Chats, Messages, Calls, Settings.
- Зв'язки між таблицями гарантують цілісність даних та дозволяють ефективно виконувати запити.

2.2.2. Вибір та обґрунтування патернів реалізації

Для побудови системи були застосовані наступні патерни:

1. Repository pattern

- Використовується для роботи з базою даних через Entity Framework Core.
- Патерн дозволяє ізолювати логіку доступу до даних від контролерів і сервісів, забезпечуючи гнучкість та легкість тестування.
- Наприклад, методи для отримання повідомлень чи чатів реалізовані у відповідних репозиторіях.

2. Singleton

- Використовується для забезпечення єдиного доступу до об'єкта конфігурації або сервісу (наприклад, налаштування JWT або підключення до бази даних).
- Гарантує, що спільні ресурси ініціалізуються лише один раз.

3. Client-Server (клієнт-серверна архітектура)

- Основний принцип побудови системи: фронтенд (клієнт) звертається до серверної частини через REST API.
- Сервер відповідає за автентифікацію, обробку даних і взаємодію з базою даних, а клієнт забезпечує інтерфейс користувача.
- Цей патерн дозволяє розділити логіку відображення і логіку обробки даних, підвищує масштабованість та безпеку системи.

2.3. Інструкція користувача

Дана система є веб-додатком, що дозволяє користувачам здійснювати обмін повідомленнями, брати участь у чатах, переглядати історію листування та редагувати особисті налаштування.

Нижче наведено покрокову інструкцію для роботи з програмою.

1. Авторизація в системі

1. Відкрийте веб-додаток у браузері.
2. У формі входу введіть:
логін, пароль.
3. Натисніть кнопку «**Увійти**».
4. Після успішної авторизації система перенаправить користувача на головну сторінку.

2. Головний інтерфейс

Після входу користувач бачить:

- список доступних чатів (зліва),
- вибраний чат та його повідомлення (справа),
- панель введення нового повідомлення.

У верхній частині також відображається ім'я авторизованого користувача.

3. Робота з чатами

1. У лівій панелі виберіть будь-який чат, натиснувши на його назву.
2. Після вибору з'явиться:
 - назва чату,
 - опис,
 - історія повідомлень.

4. Перегляд та відправлення повідомлень

1. Повідомлення відображаються у хронологічному порядку, разом із:
 - ім'ям відправника,
 - текстом,
 - часом надсилання.
2. Щоб надіслати повідомлення:
 - введіть текст у поле «Введіть повідомлення»,
 - натисніть **Enter** або кнопку «Відправити».
3. Після відправлення повідомлення миттєво з'являється у стрічці.

5. Налаштування користувача

У верхній чи боковій частині інтерфейсу доступна кнопка «Налаштування».

У цьому розділі користувач може змінити:

- колір фону інтерфейсу (за допомогою color-picker),
- розмір шрифту повідомлень.

Усі зміни зберігаються автоматично та застосовуються до інтерфейсу користувача.

ВИСНОВКИ

У ході виконання курсової роботи було розроблено та реалізовано веб-систему для обміну повідомленнями, що за своєю структурою та функціональністю наближена до сучасних месенджерів. У процесі роботи було проаналізовано вимоги до системи, спроектовано архітектуру, розроблено базу даних, створено серверну частину та клієнтський інтерфейс.

Реалізація проєкту охопила такі ключові аспекти:

- створення структурованої бази даних із використанням ORM-фреймворку Entity Framework Core;
- побудова API для взаємодії клієнта та сервера з дотриманням принципів REST;
- створення зручного та інтуїтивного інтерфейсу на основі Vue.js;
- забезпечення обробки чатів, повідомлень і налаштувань користувача;
- впровадження можливості персоналізації інтерфейсу (зміна кольору фону та розміру шрифту).

Під час реалізації було застосовано сучасні підходи до побудови клієнт–серверних систем, а також використано патерни Repository та MVC, що забезпечують масштабованість, модульність і легкість супроводу коду.

Створена система дозволяє користувачам: переглядати доступні чати, переглядати історію листування, відправляти повідомлення, а також налаштовувати вигляд інтерфейсу відповідно до власних уподобань. Отриманий застосунок є повністю працездатним, легко розширюваним і може слугувати основою для подальшого розвитку, зокрема додавання реальних аудіо- та відеодзвінків, системи сповіщень чи розширеної системи ролей користувачів.

Виконана робота підтверджує можливість побудови сучасного веб-месенджера з використанням актуальних технологій та демонструє практичні навички проєктування, програмування та інтеграції окремих компонентів у єдину систему.

ДОДАТОК А

Посилання на GitHub: <https://github.com/ShadlerAndrii/SkyChat/tree/master>

ДОДАТОК Б

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using Skype.Constants;
using Skype.Data;
using Skype.Repositories;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace Skype.Controllers
{
    [Route("[controller]")]
    [ApiController]
    public class UserDataController : ControllerBase
    {
        RepositoryUsers _repository;

        IConfiguration _configuration;

        public UserDataController(RepositoryUsers repository, IConfiguration
configuration)
        {
            _repository = repository;
            _configuration = configuration;
        }
    }
}

```

```

private string GenerateJwtToken(string id, string role, string username, string
name)
{
    var claims = new[]
    {
        new Claim("id", $"{id}"),
        new Claim(ClaimTypes.Role, $"{role}"),
        new Claim("username", $"{username}"),
        new Claim("name", $"{name}")
    };

    var secretKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));
    var loginCredentials = new SigningCredentials(secretKey,
SecurityAlgorithms.HmacSha256);

    var tokenOptions = new JwtSecurityToken
    (
        issuer: _configuration["Jwt:Issuer"],
        audience: _configuration["Jwt:Audience"],
        claims,
        expires: DateTime.UtcNow.AddMinutes(60),
        signingCredentials: loginCredentials
    );

    var tokenString = new
JwtSecurityTokenHandler().WriteToken(tokenOptions);

    return tokenString;
}

[HttpGet]

```

```
[Authorize(Roles = "Support")]
public async Task<List<User>> GetData()
{
    return await _repository.GetUserData();
}
```

```
[HttpPost]
```

```
[AllowAnonymous]
```

```
public async Task<IActionResult> AddData( [FromForm] string name,
                                           [FromForm] string username,
                                           [FromForm] string phone,
                                           [FromForm] string password,
                                           [FromForm] UserRole role)
{
    if (!await _repository.TryAddUserData(name, username, phone, password,
role))
    {
        return Conflict("User already exist");
    }

    return Ok();
}
```

```
[HttpPost("authenticate")]
```

```
[AllowAnonymous]
```

```
public async Task<IActionResult> LoginUser( [FromForm] string username,
                                           [FromForm] string password)
{
    var user = await _repository.LoginUser(username, password);
}
```

```

        if (user == null)
        {
            return Unauthorized();
        }

        var id = user.Id.ToString();
        var role = user.Role.ToString();
        var name = user.Name.ToString();

        var usernameNotLower = user.Username.ToString();

        var token = GenerateJwtToken(id, role, usernameNotLower, name);

        return Ok( new { token } );
    }
}

using Microsoft.EntityFrameworkCore;
using Skype.Constants;
using Skype.Data;
using System.Runtime.CompilerServices;
using System.Security.Cryptography;

namespace Skype.Repositories
{
    public class RepositoryUsers
    {
        AppDbContext _dbContext;
    }
}

```

```
RepositorySettings _settings;
```

```
public RepositoryUsers(AppDbContext dbContext, RepositorySettings repositorySettings)
```

```
{
    _dbContext = dbContext;
    _settings = repositorySettings;
}
```

```
private string ConvertPasswordToHash(string password)
```

```
{
    // Генеруємо salt (16 байт)
    byte[] salt = RandomNumberGenerator.GetBytes(16);
```

```
    // Рахуємо хеш PBKDF2(SHA256, 100k iterations)
```

```
    var pbkdf2 = new Rfc2898DeriveBytes(password, salt, 100_000,
    HashAlgorithmName.SHA256);
```

```
    byte[] hash = pbkdf2.GetBytes(32); // 32 байти = 256 біт
```

```
    // Зберігаємо разом: salt + hash → Base64
```

```
    byte[] hashBytes = new byte[48]; // 16 байт salt + 32 байт hash
```

```
    Buffer.BlockCopy(salt, 0, hashBytes, 0, 16);
```

```
    Buffer.BlockCopy(hash, 0, hashBytes, 16, 32);
```

```
    return Convert.ToBase64String(hashBytes);
```

```
}
```

```
private bool VerifyPassword(string savedHash, string password)
```

```
{
    byte[] hashBytes = Convert.FromBase64String(savedHash);
```



```
byte[] salt = new byte[16];
Buffer.BlockCopy(hashBytes, 0, salt, 0, 16);
```

```
byte[] storedHash = new byte[32];
Buffer.BlockCopy(hashBytes, 16, storedHash, 0, 32);
```

```
var pbkdf2 = new Rfc2898DeriveBytes(password, salt, 100_000,
HashAlgorithmName.SHA256);
```

```
byte[] hashToCheck = pbkdf2.GetBytes(32);
```

```
return CryptographicOperations.FixedTimeEquals(storedHash,
hashToCheck);
}
```

```
public async Task<List<User>> GetUserData()
{
    return await _dbContext.Users.ToListAsync();
}
```

```
public async Task<bool> TryAddUserData( string name,
                                     string username,
                                     string phone,
                                     string password,
                                     UserRole role)
{
    var existingUser = await _dbContext.Users.FirstOrDefault(u =>
u.Username == username);
    if (existingUser != null)
    {
```

```

        return false;
    }

```

```

    User newUser = new User()
    {
        Name = name,
        Username = username,
        Phone = phone,
        Password = ConvertPasswordToHash(password),
        Role = role
    };

```

```

    _dbContext.Add(newUser);
    await _dbContext.SaveChangesAsync();

```

```

    await _settings.AddSettingData(newUser.Id);

```

```

    return true;
}

```

```

public async Task<User> LoginUser( string username,
                                   string password)

```

```

{
    var user = await _dbContext.Users
        .SingleOrDefaultAsync(u => u.Username.ToLower() ==
username.ToLower());

```

```

    /*

```

```

    var a = await _dbContext.Users.Where(u => u.Username == username)
        .Include(s => s.Setting)

```

```

        .Select(s => new Setting()
        {
            FontSize = s.Setting.FontSize,
            BgColour = s.Setting.BgColour,
        })
        .ToListAsync();
    */

    if (user == null)
    {
        return null;
    }

    if (!VerifyPassword(user.Password, password))
    {
        return null;
    }

    return user;
}

//public async Task<List<Chat>> GetChatData(int userId)
//{
//    var list = await _dbContext.Users
//        .Where(u => u.Id == userId)
//        .SelectMany(u => u.Chat)
//        .ToListAsync();

```

```

        // return list;
    //}
}
}

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity.Data;
using Microsoft.AspNetCore.Mvc;
using Skype.Data;
using Skype.Repositories;

```

```

namespace Skype.Controllers
{
    [Authorize]
    [ApiController]
    public class SettingDataController : ControllerBase
    {
        RepositorySettings _repository;

        public SettingDataController(RepositorySettings repository)
        {
            _repository = repository;
        }

        [HttpGet]
        [Route("[controller]")]
        public async Task<List<Setting>> GetData()
        {
            return await _repository.GetSettingData();
        }
    }
}

```

```
[HttpGet]
[Route("[controller]/{userId}")]
public async Task<List<Setting>> GetUserData(int userId)
{
    var userSetting = await _repository.GetUserSettingData(userId);

    return userSetting;
}
```

```
[HttpPost]
[Route("[controller]")]
public async Task<IActionResult> AddData([FromForm] int userId)
{
    await _repository.AddSettingData(userId);

    return Ok();
}
```

```
[HttpPut]
[Route("[controller]")]
public async Task<OkResult> ChangeData( [FromForm] int id,
                                         [FromForm] string bgColour,
                                         [FromForm] string fontSize)
{
    await _repository.ChangeSettingData(id, bgColour, fontSize);

    return Ok();
}
}
```

```

    }

    using Microsoft.EntityFrameworkCore;
    using Skype.Data;

    namespace Skype.Repositories
    {
        public class RepositorySettings
        {
            AppDbContext _dbContext;

            public RepositorySettings(AppDbContext dbContext)
            {
                _dbContext = dbContext;
            }

            public async Task<List<Setting>> GetSettingData()
            {
                return await _dbContext.Setting.ToListAsync();
            }

            public async Task<List<Setting>> GetUserSettingData(int userId)
            {
                return await _dbContext.Setting
                    .Where(s => s.UserId == userId)
                    .ToListAsync();
            }
        }
    }

```

```

public async Task AddSettingData(int userId)
{
    Setting newSetting = new Setting()
    {
        UserId = userId,
        BgColour = "#FFFFFF",
        FontSize = "16"
    };

    _dbContext.Add(newSetting);
    await _dbContext.SaveChangesAsync();
}

public async Task ChangeSettingData(int id, string bgColour, string fontSize)
{
    Setting changedSetting = new Setting()
    {
        Id = id,
        BgColour = bgColour,
        FontSize = fontSize,
        UserId = id
    };

    _dbContext.Update(changedSetting);
    await _dbContext.SaveChangesAsync();
}
}
}

```

```

using Microsoft.AspNetCore.Authorization;

```

```

using Microsoft.AspNetCore.Mvc;
using Skype.Data;
using Skype.Repositories;
using static Skype.DTOs.ChatsDTO;

namespace Skype.Controllers
{
    [ApiController]
    [Authorize]
    public class ChatDataController : ControllerBase
    {
        RepositoryChats _repository;

        public ChatDataController(RepositoryChats repository)
        {
            _repository = repository;
        }

        [HttpGet]
        [Route("[controller]/{userId}")]
        public async Task<List<ChatWithItsUsernamesDTO>> GetData(int userId)
        {
            return await _repository.GetChatData(userId);
        }

        [HttpPost]
        [Route("[controller]")]
        public async Task<OkResult> AddData([FromForm] string name,
                                           [FromForm] string description,
                                           [FromForm] List<string> usernames)

```



```

    {
        await _repository.AddChatData(name, description, usernames);

        return Ok();
    }

    [HttpPut]
    [Route("[controller]")]
    public async Task<OkResult> ChangeData( [FromForm] int id,
                                           [FromForm] string name,
                                           [FromForm] string description,
                                           [FromForm] List<string> usernames)
    {
        await _repository.ChangeChatData(id, name, description, usernames);

        return Ok();
    }

    [HttpDelete]
    [Route("[controller]/{id}")]
    public async Task<OkResult> DeleteData(int id)
    {
        await _repository.RemoveChatData(id);

        return Ok();
    }
}
}

```

```

using Microsoft.EntityFrameworkCore;

```

```

using Newtonsoft.Json;
using Skype.Data;
using static Skype.DTOs.ChatsDTO;

namespace Skype.Repositories
{
    public class RepositoryChats
    {
        AppDbContext _dbContext;

        public RepositoryChats(AppDbContext dbContext)
        {
            _dbContext = dbContext;
        }

        private async Task<List<User>> GetListOfUsers(List<string> usernames)
        {
            //var users = await _dbContext.Users
            //    .Where(u => usersId.Contains(u.Id))
            //    .ToListAsync();

            var users = await _dbContext.Users
                .Where(u => usernames.Contains(u.Username.ToLower()))
                .ToListAsync();

            return users;
        }

        public async Task<List<ChatWithItsUsernamesDTO>> GetChatData(int
userId)

```

```

{
    return await _dbContext.Chats
        .Where(c => c.User.Any(u => u.Id == userId))
        .Select(c => new ChatWithItsUsernamesDTO
        {
            Id = c.Id,
            Name = c.Name,
            Description = c.Description,
            Usernames = c.User.Select(u => u.Username).ToList()
        })
        .ToListAsync();
}

```

```

public async Task AddChatData(string name, string description, List<string>
usernames)
{
    Chat newChat = new Chat()
    {
        Name = name,
        Description = description,
        User = await GetListOfUsers(usernames)
    };

    _dbContext.Chats.Add(newChat);
    await _dbContext.SaveChangesAsync();
}

```

```

public async Task ChangeChatData(int id, string name, string description,
List<string> usernames)
{

```

```
var chat = await _dbContext.Chats
    .Include(c => c.User)
    .SingleOrDefaultAsync(c => c.Id == id);

if (chat == null)
{
    return;
}

chat.Name = name;
chat.Description = description;

usernames = usernames
    .Select(u => u.ToLower())
    .ToList();

var currentUsernames = chat.User
    .Select(u => u.Username.ToLower())
    .ToList();

var usersToRemove = chat.User
    .Where(u => !usernames.Contains(u.Username.ToLower()))
    .ToList();

var usernamesToAdd = usernames
    .Except(currentUsernames)
    .ToList();

var usersToAdd = await _dbContext.Users
    .Where
```

```

        (
            u => usernamesToAdd
                .Contains(u.Username.ToLower())
        )
        .ToListAsync();

    foreach (var user in usersToRemove)
    {
        chat.User.Remove(user);
    }

    foreach (var user in usersToAdd)
    {
        chat.User.Add(user);
    }

    await _dbContext.SaveChangesAsync();
}

public async Task RemoveChatData(int id)
{
    await _dbContext.Database
        .ExecuteSqlRawAsync
        (
            "DELETE FROM ChatUser WHERE ChatId = {0}",
            id
        );

    await _dbContext.Messages
        .Where(m => m.ChatId == id)

```

```

        .ExecuteDeleteAsync();

        await _dbContext.Chats
            .Where(c => c.Id == id)
            .ExecuteDeleteAsync();
    }
}

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Skype.Data;
using Skype.Repositories;
using static Skype.DTOs.MessagesDTO;

namespace Skype.Controllers
{
    [ApiController]
    [Authorize]
    public class MessageDataController : ControllerBase
    {
        RepositoryMessages _repository;

        public MessageDataController(RepositoryMessages repository)
        {
            _repository = repository;
        }

        [HttpGet]
        [Route("[controller]/{chatId}")]

```

```

        public async Task<List<MessageWithItsUserNameDTO>> GetData(int
chatId)
        {
            return await _repository.GetMessageData(chatId);
        }

```

```

[HttpPost]
[Route("[controller]")]
public async Task<ActionResult> AddData( [FromForm] int chatId,
                                         [FromForm] int ownerId,
                                         [FromForm] string text)
{
    await _repository.AddMessageData(chatId, ownerId, text);

    return Ok();
}
}
}

```

```

using Microsoft.EntityFrameworkCore;
using Skype.Data;
using static Skype.DTOs.MessagesDTO;

```

```

namespace Skype.Repositories
{
    public class RepositoryMessages
    {
        AppDbContext _dbContext;

        public RepositoryMessages(AppDbContext dbContext)

```

```
{
    _dbContext = dbContext;
}
```

```
public async Task<List<MessageWithItsUserNameDTO>>
GetMessageData(int chatId)
{
    var list = await _dbContext.Messages
        .Where(m => m.ChatId == chatId)
        .Join(
            _dbContext.Users,
            message => message.OwnerId,
            user => user.Id,
            (message, user) => new MessageWithItsUserNameDTO
            {
                Id = message.Id,
                ChatId = message.ChatId,
                OwnerId = message.OwnerId,
                TimeSend = message.TimeSend,
                Text = message.Text,

                OwnerName = user.Name,
            }
        )
        .ToListAsync();

    return list;
}
```

```
public async Task AddMessageData(int chatId, int ownerId, string text)
```



```
{
    Message newMessage = new Message()
    {
        ChatId = chatId,
        OwnerId = ownerId,
        TimeSend = DateTime.UtcNow,
        Text = text
    };

    _dbContext.Messages.Add(newMessage);
    await _dbContext.SaveChangesAsync();
}
}
```